INDEX

NAME: Nikhil Vinayak P STD.: III SEC.: CSE FD ROLL NO.: 240701359 SUB.: Computer Networks Observation

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.	20/07/2026	Basic Networking Commands		
2.	27/07/2026	Different types of network cable		
3	30/07/2026	Cisco Packet Tracer		
4	03/08/2026	Setting Up LAN		
5	10/08/2026	Wireshark		
6	07/09/2026	Error Detection & Correction Using Hamming Code		
7	08/09/2026	Flow Control Using Sliding Window Protocol		
8	22/09/2026	NMAP using Tryhackme		
9	28/09/2026	Subnetting in Cisco Packet Tracer Simulator		
10	28/09/2026	Internetworking in Cisco Packet Tracer Simulator		
11	29/09/2026	End-to-end communication at Transport Layer		
12	29/09/2026	Implement your own ANS program		
13	5/10/2026	Packet Sniffing RAW Sockets		

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
14	5/10/2026	a) Static Routing b) RIP (Routing Information Protocol)		

Completed by Mahesh

DATE: 20/07/2026

Practical - 1

Basic Networking Commands

AIM:

Study of various network commands used in Linux and Windows

COMMANDS:

1. arp -a: ARP is short form of Address Resolution Protocol. It will show the IP address of your computer along with the IP address and MAC address of your router.

OUTPUT:

172.16.8.149	00:27:0e:13:f3:fe	enp3s0
172.16.9.212	7c:57:58:38:da:a3	enp3s0
172.16.8.186	7c:57:58:38:e6:24	enp3s0
172.16.8.249	00:27:0e:13:f5:77	enp3s0
172.16.11.207	7c:57:58:38:46:87	enp3s0

2. hostname: This is the simplest of all TCP/IP commands. It simply displays the name of your computer.

OUTPUT: localhost.localdomain

3. netstat: (network statistics) netstat displays a variety of statistics about a computers active TCP/IP connections. It is a command line tool for monitoring network connections both incoming and outgoing as well as viewing routing tables, interface statistics, etc

Eg: netstat -r

OUTPUT:

Proto	Local Address	Foreign Address	State
tcp	localhost.localdoma:ssh	172.16.8.135:56449	ESTABLISHED
tcp	localhost.localdoma:ssh	172.16.11.207:59129	ESTABLISHED
tcp	localhost.localdoma:ssh	172.16.8.140:52171	ESTABLISHED
tcp	localhost.localdoma:ssh	172.16.8.166:62861	ESTABLISHED
tcp	localhost.localdoma:ssh	172.16.8.162:50007	ESTABLISHED

4. nslookup: (name server lookup) is a tool used to perform DNS lookup in Linux. It is used to display DNS details, such as IP address of particular computer, Me MX records for a domain or NS servers of a domain. nslookup can operate in two modes: interactive and non-interactive.
eg: nslookup www.google.com

OUTPUT:

```
Server: 172.16.8.1
Address: 172.16.8.1#53

Non-authoritative answer:
Name: www.google.com
Address: 142.251.156.119
Name: www.google.com
Address: 142.251.152.119
```

5. ping: (packet internet groper) command is the best way to test connectivity blw two nodes. Ping use ICMP (Internet Control Message Protocol) to communicate to other devices.

1. #ping hostname (ping localhost)
2. #ping ip address (ping 4.2.2.2)
3. #ping fully qualified domain name (ping www.facebook.com)

OUTPUT:

```
PING localhost (localhost (::1)) 56 data bytes
64 bytes from localhost (::1): icmp-seq=1 th=64 time=0.81ms
64 bytes from localhost (1:1): icmp-seq=2 th=64 time=0.051ms
64 bytes from localhost (::1): icmp_seq=3 th=64 time=0.044ms
--- localhost ping statistics ---
3 packets transmitted, 3 received, 0.1 packet loss, time 2040ms
rtt min/avg/max/mdev = 0.045/0.055/0.075/0.015 ms
```

6. route: It is used to show/manipulate IP routing table. It is primarily used to setup static routes to specific host or networks via an interface.

OUTPUT:

Kernel IP routing table

Destination	Gateway	Genmsk	Flags	Metric	Ref	Use	Iface
default	gateway	0.0.0.0	un	100	0	0	enp3s0
172.16.8.0	0.0.0.0	255.255.252.0	U	100	0	0	enp3s0

7. ip config: Configs displays detailed configuration information about TCP/IP configuration including Router, Gateway, DHCP, DNS.

OUTPUT:

Ethernet adapter Ethernet 2:

Connection specific DNS suffix:

Link local IPv6 Address : fe80:f119:4ed6

IPv4 address : 192.168.56.1

Subnet Mask : 255.255.255.0

Default Gateway:

LINUX NETWORKING COMMANDS:

1. ip The ip command is one of the basic commands every administrator will need in daily work from setting up new system and assigning IPs to troubleshooting existing system. The ip command can show address information manipulate routing. plus display network various devices, interfaces, and tunnels.

ip <OPTIONS> <OBJECT> <COMMAND>

Commands to use for ip command

a) To show IP address assigned to an interface on your server.

ip address show

o/p 1: Lo: <LOOPBACK, UP, LOWER_BACK> mtu 65536 qdisk noqueue
State UNKNOWN group default qun 1000
Link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
inet 127.0.8.1/8 scope host LO
valid_yt forever Pseferred_lft forever
inet6 ::1/128 scope host
valid_lft forever preferred_lft forever
2: enpos3: <BROADCAST.MULTICAST.UP.LOWER_UP> mn 1500
qdin fq_codu state VP group default qlen 1000
Link/ether 08:00:27:tf:sa:4f brd f6:f6:ff:ff:ff:ff
inet 10.0.2.15/24 brd 10.0.2.255 supe-gwbal dynamnic

2. ipconfig: This command is staple in many sysadmins tool best for configuring & troubleshooting networks. It has been replaced by ip command discussed.

o/p enpos3: flags = 4163 <UP, BROADCAST, RUNNING>
mn 1500 inet 10.0.2.15 netmask 255.255.255.0
broadcast 10.0.2.255

3. mtr: Its a program with command line interface that servers as network diagnostic & troubleshooting tool. It combines ping and tracerate commands.

mtr <options> hostname/IP

a) mtr

	Host	Loss%	Snt	Last	Avg	Best	Wrst	Stdev
1.	localhost	0.0%	26	0.1	0.1	0.1	0.3	0.0

b) mtr google.com

O/P

	Host	Loss%	Snt	Last	Avg	Best	Wrst	Stdev
1.	_gateway	0.0%	46	0.6	0.7	0.3	1.2	0.3
	:							
12.	lcmaa-ao	72%	72	92.8	465.1	66.3	1781	949.4

4. tcpdump: Its designed for capturing and displaying packets. Install: dnf install -y tcpdump

O/P: dropped privs to tcpdump

tcpdump: verbose rp supressed, Use -v[v] ...

snapshot length 262144 bester

5. ping: Tool that verifies IP-level connectivity to another TCP/IP computer by sending ICMP ECHO request message. Ping primarily used for troubleshooting reachability and name resolution.

i) ping google.com

O/P PING google.com (142.251.223.14) 56(84) bytes

BASIC NETWORKING COMMANDS:

8. nbtstat -a helps solve problems with NetBIOS name resolution.

⊙ IP

NBTSTAT [[-a RemoteName] [A IP address] [-c] [-n] [-r] [-R] [-RR] [-s] [-S] [interval]]

9. pathping: Its a combination of ping and Tracert commands. Pathping traces route to destination address.

⊙ IP

pathping [-g host-list] [-h maximum-hops] [-i address] [-n] [-p period] [-q num-queries] [-w timeout] [-4] [6] target-name

RESULT:

Thus the various network commands used in Linux and windows was executed successfully.

DATE 27/7/2026

Practical - 2

Understanding different types of network cable

AIM

Study of different types of Network cables.

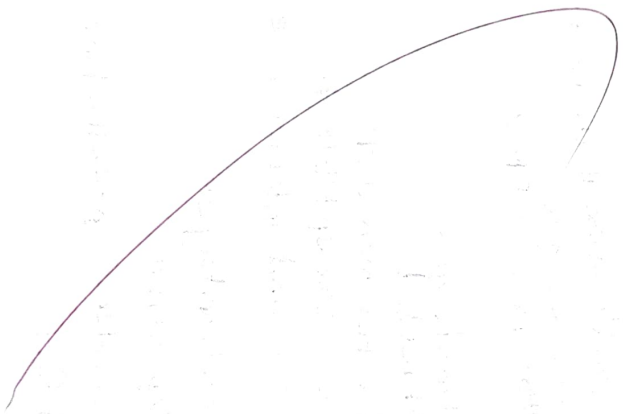
Different types of cables used in networking are:

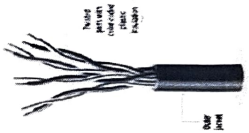
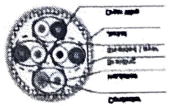
1) Unshielded Twisted Pair (UTP) cable

2) Shielded Twisted Pair (STP) cable

3) Coaxial Cable

4) Fibre optic cable



Cable Type	Maximum Data Transmission	Advantages / Disadvantages	Application Use	Image
UTP	Category 3 10 bps Category 5 upto 100mbps Category 5e 1Gbps	Advantages: • Cheaper in cost • Easy to install - small overall diameter Disadvantages: • More prone to EMI	10 Base-T ethernet Fast Ethernet, Gigabit Ethernet	
STP SSTP	Category 6, 6a 10Gbps Category 7 10Gbps	Advantages: • Shielded • Faster than UTP • Less susceptible to noise and interference Disadvantages: • Expensive • Greater installation effort	Gigabit ethernet 10G ethernet (55m) Widely used in data centers Gigabit ethernet 10G Ethernet (100m)	 

Coaxial Cable	RG-6 RG-59 RG-11 100-100 Mbps	Advantages: • High bandwidth • Immune to interference • Low loss bandwidth • Versatile Disadvantages: • Limited distance • Cost • Size is bulky	Speed of signal is 500m Television network High speed internet connection	 
Fibre optics cable	Single mode Multi mode 100 Gbps	Advantages: • High speed • High bandwidth • High security • long distance Disadvantages: • Expensive • Requires skilled installers	Maximum distance of fibre optics cable is around 100 m	

RESULT

Thus, we have studied the different types of network cables.

DATE: 30/07/2026

Practical - 3

Cisco Packet Tracker

AIM:

To study the packet tracer tool installation and user interface overview.

PROCEDURE:

Analyse the behavior of network devices using CISCO PACKET TRACKER simulator.

1. From network component box, click and drag-drop below components.
   a) 4 Generic PCs and one hub
   b) 4 Generic PCs and one switch

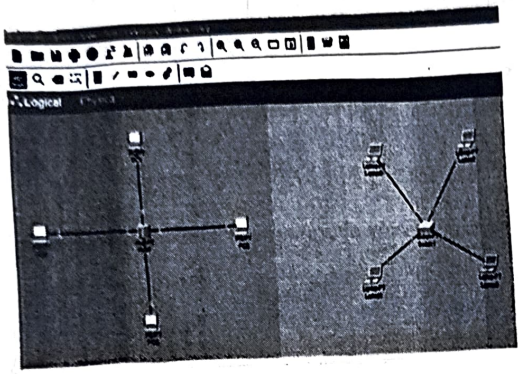
2. Click on connections
   a) Click on copper straight-through cable
   b) Select one of the PC and connect it to HUB using cable. The link LED should glow in green, indicating link is up. Connect other PCs to HUB.
   c) Similarly connect 4PCs to switch using copper straight through cable.

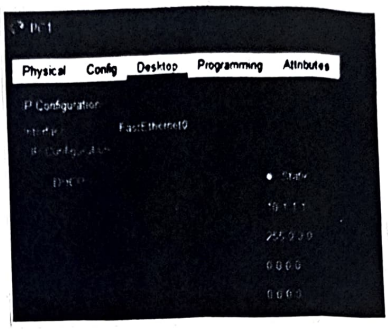
3. Click on PC's connected to Hub, go to desktop. Click IP configuration, set IP, subnet mask. Default gateway and DNS server information is not needed as only 2 end devices in network.
Click on the PDU (msg icon) from common tool bar - drag and drop it on PC (source machine) and drop it on another PC (destination machine) connected to the hub.

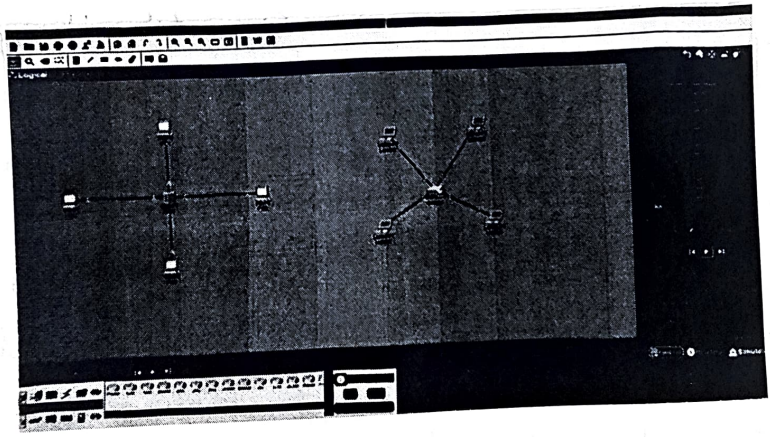
4. Observe the PDU from source PC to destination PC by selecting realtime mode of simulation.

5. Repeat step 3 to step 5 for PCs connected to the switch.

6. Observe how hub and switch are forwarding the PDU and write conclusion about the behaviour of switch and hub.







RESULT:

Thus, we have analysed behaviours of network devices using CISCO packet tracer simulator.

DATE: 03/08/2026

Practical - 4

Setting Up LAN

AIM To design, setup and configure LAN connecting 4 computers through network switch assign static IP address, verify connectivity using ping and demonstrate data sharing.

EQUIPMENT REQUIRED:

S No	Equipment	Qty	Specifications
1.	Personal Computer	4	Ethernet (NIC) enabled
2.	Network switch	1	8/16/24 port unmanaged/ managed
3.	Ethernet Cables	4	Straight - through RS 45 connectors

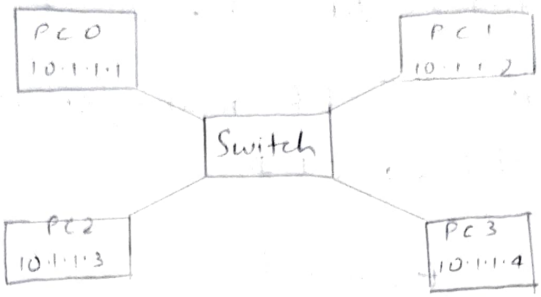
THEORY:

LAN is a network that connects both computer and devices within limited geographical area allowing them to share resources such as files, printers and internet.

Star topology is used here, where all PCs are individually connected to a central switch. The switch operates at layer 2 of OSI model and forwards data frames b/w connected devices based on their MAC addresses.

NETWORK DEVICES

LAN was implemented using Star Topology



IP ADDRESS CONFIGURATION TABLE:

Device	IP Address	Subnet Mask	Connected to
PC0	10.1.1.1	255.0.0.0	Switch Port Fa0/1
PC1	10.1.1.2	255.0.0.0	Switch Port Fa0/2
PC2	10.1.1.3	255.0.0.0	Switch Port Fa0/3
PC 3	10.1.1.4	255.0.0.0	Switch Port Fa0/4
Switch (Management)	10.1.1.5	255.0.0.0	VLAN 1 Interface

IMPLEMENTATION IN CISCO PACKET TRACER:

Step 1 - Place devices

1. Open Cisco
2. From device category select end devices & drag 4 PCs
3. Select switches & drag 1 switch (2960 switch)

Step 2 - Cable the Devices

4. Click connections → select copper straight through cable
5. Click each PC, then click switch, select an available fast ethernet port.
6. Repeat all 4PC (ports Fa0/1 to Fa0/4). Green light confirm successful connection.

Step 3 - Assign IP Address to PCs

7. Click PC → Desktop tab → IP configuration
8. Enter static IP address of subnet mask

Step 4 - Configure switch management IP

9. Click switch → CLI

10. enable
configure terminal
interface vlan 1
IP address 10.1.1.5 255.0.0.0
no shutdown
exit

Step 5 - Verify connectivity

11 Click PC → Desktop → Command Prompt
12. Use ping command to test connectivity to their PC
13. Successful 'Reply From ...' message

Step 6 - Demonstrate Data Sharing

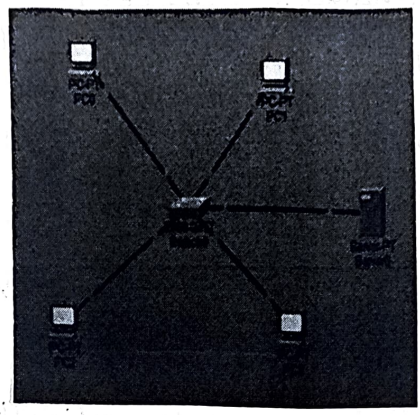
14. Add server device (End device → server PT) and connect to switch
15. Assign IP address via IP configuration
16. Open config tab enable HTTP service
17. Open web ~~server~~ browser enter server IP address to load page.

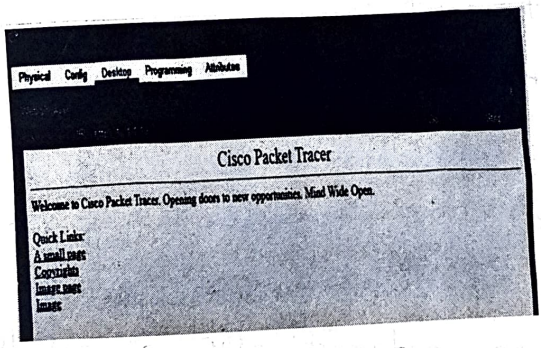
Step 7 - Observe Packet Flow

18. Switch to Simulation mode
19. Send ping or web request btw two devices and use play/capture control to step through packet.

OBSERVATION:

All 4 PC successfully connected to the switch in star topology and assigned unique static IP addresses within same subnet (10.0.0.0 | 255.0.0.0). The shared folder successfully accessed ~~CUTE~~ LAN was function correctly for resource sharing.





RESULT:

The LAN was successfully designed and configured. Connectivity blw all PC's verified using PING command and resource sharing was successfully demonstrated.

DATE: 10/08/2026

Experiment - 5

Wireshark

AIM:

Experiments on Packet capture tool - Wireshark

Packet Sniffer:

Sniffs messages being sent/received from by your computer.

Capturing and analysing packets:

To filter, capture, view packets in wireshark tool. Capture 100 packets from the Ethernet.

PROCEDURE:

→ Select Local Area Connection in Wireshark
→ Go to capture option
→ Select stop capture automatically after 100 packets
→ Click start capture
→ Save packets

1. To display TCP/UDP packet

→ Select LA connection in wireshare
→ Go to capture option
→ Stop capture automatically after 100 packets
→ Click start capture
→ Select TCP packets
→ To see flow graph
→ Save packets

2. To display ARP packets

→ Go to capture option

→ stop capture

→ Click start

→ Search ARP packet in search bar

→ Save packet

3. To display DNS packet

→ Go to capture

→ Select stop capture after 100 packet

→ Click start

→ Search DNS packet

→ See flow graph click statistics, flow graph

→ Save packets

4. To display HTTP packet

→ Select LA connection in wireshark

→ Click start capture

→ Search HTTP packet in search bar

→ Save packet

5. To display IPl ICMP packet

→ Select LA connection

→ Click start capture

→ Save packet

6. To display DHCP packet

→ Select LA connection

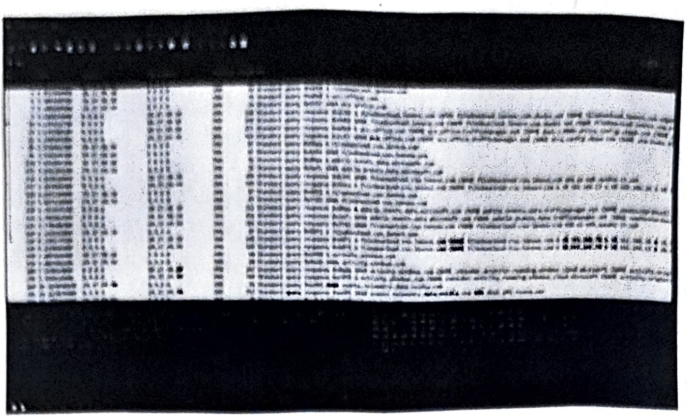
→ Go to capture option

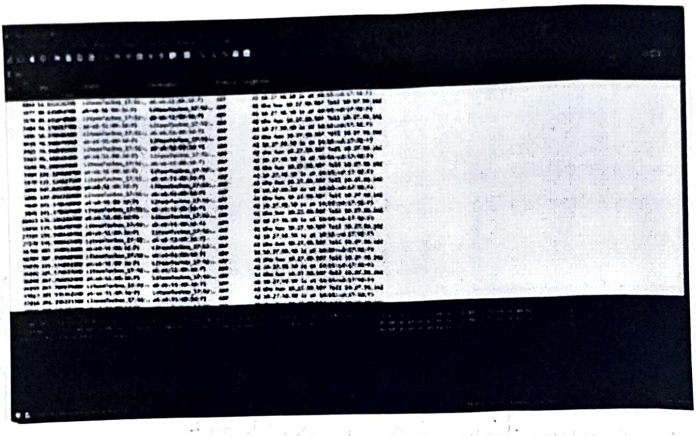
→ Select stop capture after 100 packet

→ Click start capture

→ Search DHCP packet in search bar

→ Save packet





RESULT:

Thus, TCP/UDP, ARP, DNS, HTTP, IP/ICMP, DHCP packets was successfully executed.

DATE : 07/09/2026

Experiment - 6

Error Detection and Correction Using Hamming Code

AIM: To implement error detection & correction using hamming code concept. Make a test run to input data stream and verify error correction.

SENDER:

*Input to sender file shld be a text of any length, prgrm should convert the text to binary.

*Apply hamming code concept on the binary data and add redundant bits to it

*Save this output in a file called channel.

CODE:

```
data = input("Enter 4 bits: ")
d1, d2, d3, d4 = map(int, data)
P1 = d1 ^ d2 ^ d4
P2 = d1 ^ d3 ^ d4
P4 = d2 ^ d3 ^ d4
code = [P1, P2, d1, P4, d2, d3, d4]
print("Sender hamming code: ", "join
      (map(str, code)))
```

RECEIVER SIDE :-

* Receive the Hamming code from the sender.
* Enter the received code
* Calculate the parity check values
* Find the error position using parity values
* Correct the error by changing bit

CODE :-

```
r= list (map(int,input("Enter received 7 bits: ")))
r1=r[0]^ r[2]^r[4]^ r[6]
r2=r[1]^ r[2]^r[5]^ r[6]
r3=r[3] ^r[4]^r[5]^ r[6]
error=r1*1+r2*2+r4*4
if error ==0:
    print ("No error")
else:
    print("Error at position: ", error)
    r[error-1]^ =1
print("Corrected code: ", ".join(map(str, received)))
```

INPUT: Enter 4 data bits : 1011

Sender Hamming Code : 0110011

Received 7 bits : 0010011

OUTPUT: Error at position: 2

Corrected Code : 0110011

RESULT:

Thus, Error detection & Error correction using Hamming code was executed successfully.

DATE: 8/9/2026

Experiment 7

Flow Control Using Sliding Window Protocol

AIM:

To implement flow control at data link layer using SLIDING WINDOW PROTOCOL.

SENDER:

* Get the wndw size from user
* Get txt msg
* Dvd the msgs → frames
* Send to SENDER_BUFFER
* Wait for acknowledgement
* Read RECEIVER_BUFFER
* If ACK is received, send the next
* If NACK is received, retransmit the frame

RECEIVER:

* Read frames from S_B
* Check frame no.
* If the frame no. is expected, send ACK
* Otherwise, NACK
* Write ACK/NACK into R_B

SENDER CODE:

```
import time
w=int(input("Window size: "))
msg = input("Msg: ")
i=0
while i<len(msg):
    end=min(i+w, len(msg))
    with open("SENDER_BUFFER", "w") as f:
        for j in range(i, end):
            f.write(f"{j}:{msg[j]}\n")
            print("Frame ", j, "→", msg[j])
    time.sleep(2)
    with open("RECEIVER_BUFFER") as f:
        ack=f.read()
    i=int(ack[1])
print("Transmission completed")
```

RECEIVER CODE:

```
import time
import os
expected=0
while true:
    if not os.path.exists("Sender_Buffer"):
        continue
    with open("Sender_Buffer") as f:
        frames=f.readlines()
    for x in frames:
        n=int(x.split(":")[0])
        if n==expected:
            expected+=1
            print("Received frame", n)
            with open("Receiver Buffer", "w")
            as f:
                f.write("ACK"+str(expected))
```

```
        else:
            with open("Receiver_Buffer","w")
            as f:
                f.write("NACK "+str(expected))
            break

        time.sleep(1)
```

Input: Enter window size : 3
Enter text message : HELLO

Output: Total Frames : 5
Msg : HELLO
Saving Frames
Frame No : 0 DATA : H
Frame No : 1 DATA : E
Frame No : 2 DATA : L

Frames saved to Sender-buffer
Waiting for Receiver ACK...

Receiver Response : ACK = 2
ACK received for FRAME : 2
Sliding window moved : Next
Frame : 3
Sending Frames
Frames saved
Waiting for Receiver ACK...
Receiver Response : ACK = 4
ACK received for Frame : 4
All frames successfully sent

RESULT:

The SLIDING WINDOW PROTOCOL was implemented Successfully. ACK/NACK was simulated successfully.

DATE 22/09/2026

Experiment - 8

NMAP to discover live hosts using NMAP Scans on the tryhackme platform

AIM:

To discover live hosts using NMAP scans (ARP, ICMP, TCP/UDP) on the Tryhackme platform.

PROCEDURE:

Task 1 ⇒ *ARP scan: Uses ARP request to discover live hosts
*ICMP scan: Uses ICMP request to identify live hosts
*TCP/UDP live PING scan: sends packets to TCP ports & UDP ports to determine live hosts

Task 2 - Subnetworks

Network Segment: collection of computers linked together thru a common "medium"

Subnetwork: It is usually the equivalent of one or more segments linked together and configured to utilize the same router in an IP network.

*Subnets with /16
*Subnets with /24

Task 3 - Enumerating Targets

We need to identify the targets we want to scan before we conduct any "action"

① LIST

<machine-IP> <scanme.nmap.org>

<example> → will scan 3 IP addresses

② RANGE

→ 10 11 12 15-20

→ will scan 6 IP addresses

③ SUBNET

→ <machine-IP>/30

→ will scan 4 IP addresses

(1) NMAP HOST DISCOVERY USING ARP

nmap -PR -sn TARGETS

(2) USING ICMP

nmap -PE -sn TARGETS

(3) TCP SYN PING

nmap -PS80 TARGETS

(4) TCP ACK PING

nmap -PA80 TARGETS

(5) UDP PING

(6) NMAP PING SCAN

nmap -sn TARGETS

(7) REVERSE DNS LOOKUP

nmap -n TARGETS

nmap -R TARGETS

RESULT:

Using Tryhackme platform, we learned and executed to discover live hosts on NMAP scans (ARP, ICMP, TCP/UDP)

DO : 28/09/2026

Experiment - 9

Subnetting in Cisco Packet Tracer Simulator

AIM:

To implement classless IP subnetting using Cisco Packet Tracer & verify connectivity blw diff Subnets.

PROCEDURE:

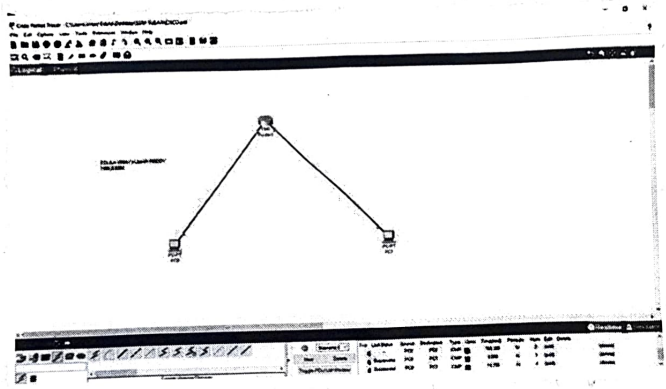
1. Open CPT & create a new network topology
2. Add routers, switches & PCs to the workspace
3. Connect the devices using appropriate cables
4. Divide the network 192.168.1.0/24 into smaller subnets using the 127 subnet mask.
5. Configure the IP addresses & subnet mask for the router interfaces
6. Configure the PC's with suitable IP addresses, subnet masks and default gateways.
7. Configure the switch ports & router interfaces
8. Use the PING cmd. to test connectivity blw devices
9. Verify that the packets are successfully transmitted blw the connected networks.

Advantages:

* Efficient use of IP address
* Reduces network congestion
* Improves network mgnt.

IP addressing

Network	Subnet mask	Router IP	PC IP Range
192.168.1.0/27	255.255.255.224	192.168.1.1	192.168.1.11 - 15
192.168.2.0/27	255.255.255.224	192.168.2.1	192.168.2.11 - 15
192.168.3.0/27	255.255.255.224	192.168.3.1	192.168.3.11 - 15
192.168.4.0/27	255.255.255.224	192.168.4.1	192.168.4.11 - 15



RESULT:

The classless IP subnetting was successfully implemented using CISCO PACKET TRACER.

DATE : 28/09/2026

Experiment - 10

Internetworking with routers in CISCO PACKET TRACER Simulator

AIM:

To internetwork with routers in CISCO PACKET TRACER Simulator

PROCEDURE:

Step 1 Configuring Router1

(i) Select the router & Open CLI

(ii) Press ENTER to start configuring Router 1

(iii) Type enable to activate the privileged mode

Step 2 Configuring PCs

(i) Assign IP to every PC in the network

(ii) Select the PC, go to desktop & select IP config. & assign an IP, default gateway, subnet mask

(iii) Assign default gateway of PC0 as 192.168.10.1

(iv) Assign default gateway of PC1 as 192.168.20.1

Step 3 : Connecting PCs with Router

(i) Connect FastEthernet 0 port of PC0 with FE0/0 port of Router 1 using a Copper straight-through cable.

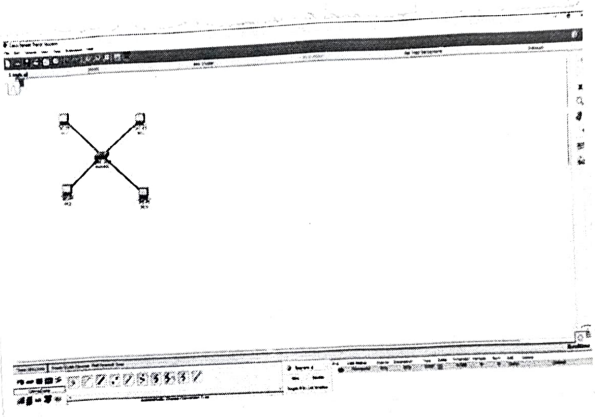
(ii) Connect FE0 port of PC1 with FE0/1 port of Router1 using a Copper straight-through cable.

Router Configuration Table:

Device Name	IP Address FE 0/0	Subnet Mask	IP Address FE 0/1	Subnet Mask
Router 1	192.168.0.1	255.255.255.0	192.168.20.1	255.255.255.0

PC Configuration Table:

Device Name	IP Address	Subnet Mask	Gateway
PC0	192.168.10.2	255.255.255.0	192.168.10.1
PC1	192.168.20.2	255.255.255.0	192.168.20.1



RESULT:

The internetworking with router was successfully implemented using CISCO PACKET TRACER.

DATE: 28/09/2026

Experiment - 11

(a) Static Routing

AIM:

To simulate and configure static routing with primary and backup routes using Cisco Packet Tracer and verify connectivity between diff nodes

PROCEDURE:

1) Open Cisco packet tracer and create the given topology with Router0, Router1, Router2, switches and PCs.
2) Configure the IP addresses on all the router interfaces and PC's according to the given topology.
3) Open the CLI of Router 0 and enter configuration mode.
4) Configure static routes for network 30.0.0.0/8 and 50.0.0.0/8 using Router1 as the primary route and Router 2 as the backup route.
5) Configurate a separate host route for 30.0.0.100/32 with Router2 as the primary route and Router1 as the backup route.
6) On Router1, configure static routes for networks 10.0.0.0/8 and 40.0.0.0/8, with primary and backup routes.
7) On Router2, configure static routes for networks 10.0.0.0/8 and 30.0.0.0/8
8) Use the command: show ip route static to verify the configured static routes.
9) Use the traceart command from a PC to verify the path taken by packets to the destination network.

10) Disconnect the primary link blw R_0 & R_1 and use traceart again to verify that the backup route becomes active.

11) Static routes can be removed using no ip route

12) Verify the routing table again using show ip route static.

Input:

```
C:\>ping 30.0.0.100
Pinging 30.0.0.100 with 32 bytes of data
Request timed out.
Reply from 30.0.0.100: bytes = 32 time=2ms TTL = 126
Reply from 30.0.0.100: bytes = 32 time = 16ms TTL=126
Reply from 30.0.0.100: bytes = 32 time=24ms TTL=126
Ping statistics
    Packets: Sent=4, Received=3, Lost=1 (25% loss),
Approx. round trip times in milli-seconds:
    Min = 2ms, Max=24ms, Avg = 14 ms

C:\>traceart 30.0.0.100
Tracing route to 30.0.0.100 over a max. of 30 hops:

  1   0ms    0ms    0ms   10.0.0.1
  2   3ms    1ms    1ms   20.0.0.2
  3   0ms    0ms    1ms   30.0.0.100

Trace complete.
```

RESULT:

Thus, the routing with primary & backup routes was successfully configured and simulated using the CISCO Packet Tracer.

Experiment - 11

(b) RIP (Routing Information Protocol)

AIM:

To simulate and configure RIP (Routing Information Protocol) using Cisco Packet Tracer and verify dynamic routing blw diff networks.

PROCEDURE:

1) Open Cisco and create given topology with Router0, Router1, Router 2, PC0 and PC1.

2) Assign IP addresses to PCs & router interfaces according to table

3) Configure the FastEthernet & serial interfaces of all routers using the assignment IP addresses.

4) Use no shutdown to activate all router interfaces.

5) For DCE serial interfaces, configure the required clock rate and bandwidth.

6) Configure RIP on R0 using :-

```
router rip
network 10.0.0.0
network 192.168.1.252
```

7) Configure RIP on R1 using :-

```
router rip
network 192.168.1.244
network 192168.1.248
```

8) Configure RIP on R2 using :-

```
router rip
network 20.0.0.0
network 192.168.1.252
network 192.168.1.244
```

9) Allow RIP to exchange routing information automatically blw the routers.

10) From PC0, use the ping command to verify connectivity with PC1.

11) Use the tracert command to observe the path followed by packets blw the networks.

12) Disconnect one of the available routes and use tracert again to verify that RIP automatically selects the alternate one.

INPUT1:
Ping 20.0.0.1

OUTPUT:
Pinging 20.0.0.1 with 32 bytes of data:

Reply from 20.0.0.1: bytes=32, time<1ms TTL=255
Reply from 20.0.0.1: bytes=32 time<1ms TTL=255
Reply from 20.0.0.1: bytes=32 time<1ms TTL=255
Reply from 20.0.0.1: bytes=32 time<1ms TTL=255

INPUT 2:
Ping 10.0.0.2

OUTPUT:
Pinging 10.0.0.2 with 32 bytes of data:

Reply from 10.0.0.2: bytes=32 time=19ms TTL=126
Reply from 10.0.0.2: bytes=32 time=19ms TTL=126
Reply from 10.0.0.2: bytes=32 time=18ms TTL=126
Reply from 10.0.0.2: bytes=32 time=19ms TTL=126

Ping statistics for 10.0.0.2:
Packets: Sent=4, Received=4, Lost=0 (0% loss)
Approx. round trip times in milli-seconds:
Min=18ms, Max=19ms, Avg=18ms

tracert 10.0.0.2

Tracing route to 10.0.0.2 over a max of 30 hops.

1	0ms	0ms	0ms	20.0.0.1
2	11ms	13ms	12ms	192.168.1.254
3	9ms	5ms	5ms	10.0.0.2

Trace complete.

RESULT:

Thus, RIP was successfully configured and simulated using CISCO Packet Tracer.

DATE : 29/09/2026

Experiment - 12

End-to-End Communication At Transport Layer

a) Implement ECHO client server using TCP sockets
b) Implement a chat prgrm using socket prgrmming

AIM:

To implement an ECHO SERVER appln using TCP sockets.

TCP

CODE:

ECHO SERVER

```
import socket
import threading
s = socket.socket()
s.bind(("Localhost", 5000))
s.listen(1)
c, addr = s.accept()
msg = c.recv(1024).decode()
print("client : ", msg)
c.send(msg.encode())
c.close()
s.close()
```

ECHO CLIENT

```
import socket
s = socket.socket()
s.connect(("Localhost", 5000))
msg = input("Enter msg : ")
s.send(msg.encode())
print("Echo : ", s.rev(1024).decode())
s.close()
```

Input:

Enter msg : Hello Server

Output:

'Server

Client connected

Received : Hello server

Client

Echo from Server : Hello Server

UDP CODE:

```
import socket
import threading
def udp_server():
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    s.bind(('127.0.0.1', 12000))
    print("[SERVER] Running on 127.0.0.1:12000")
    while True:
        data, addr = s.recvfrom(1024)
        msg = data.decode()
        if msg.lower() == 'quit':
            print(f"[SERVER] Client {addr} ended the conn.")
            break
        print(f"[SERVER] Received from {addr}: {msg}")
        s.sendto(data, addr) #Echo back
    s.close()
    print("[SERVER] Stopped.")
```

```
def udp_client():
    c=socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    server = ('127.0.0.1', 12000)
    while True:
        msg=input("Enter msg (or 'quit' to stop): ")
        c.sendto(msg.encode(), server)
        if msg.lower()=='quit':
            print('[CLIENT] Closing connection...') break
        data, _ = c.recvfrom(1024)
        print("Echo from Server: ", data.decode())
    c.close()
    print("[CLIENT] Stopped.")

if __name__=="__main__":
    threading.Thread(target=udp_server, daemon=True).
                start() udp_client()
```

Input:

Enter msg: Hello Server

Output:

Server

[SERVER] Running on 127.0.0.1:12000hi

[SERVER] Received from ('127.0.0.1', 49670): hi

Echo from Server: hi

Enter msg: quit

[CLIENT] Closing conn....[SERVER] Client('127.0.0.1', 49670) ended

[CLIENT] Stopped. [SERVER] Stopped.

RESULT:

The chat client server applications were successfully studied and implemented using TCP/UDP sockets.

DATE: 29/09/2026

Experiment - 13

Implement your own PING program.

AIM:

To implement a ping program using python to check the connectivity blw source and destination host.

ALGORITHM:

1. Get the dest^n IP from user
2. Send request to destination
3. Wait for response
4. Calculate response time
5. Display whether the host is reachable
6. Repeat the process for few request.

CODE:

```
import os
import platform
import subprocess
def ping(host="google.com", count=4):
    param="-n" if platform.system().lower()=="windows" else "-c"
    command=["ping", param, str(count), host]
    try:
        output=subprocess.check_output(cmd, universal_newlines=True)
        print(output)
    except Exception as e:
        print(f"Ping failed: {e}")
if __name__=="__main__":
    ping("8.8.8.8")
```

OUTPUT:

```
Pinging 8.8.8.8 with 32 bytes of data:
Reply from 8.8.8.8: bytes=32 time=4ms TTL=119
Reply from 8.8.8.8: bytes=32 time=9ms TTL=119
Ping statistics for 8.8.8.8:
    Packets: Sent=2, Received=2, Lost=0 (0% loss),
Approx. round trip times in milli-seconds:
    Min.=4ms, Max=9ms, Avg=7ms
```

RESULT:

The ping program was successfully studied and implemented.

DATE: 03/10/2026

Experiment - 14

Packet Sniffing Using RAW Sockets

AIM:

To capture and display network packets using RAW sockets.

ALGORITHM:

1. Create RAW socket
2. Bind the socket to the network interface
3. Receive packets from the network
4. Extract the IP header information
5. Display the source IP, destination IP and protocol
6. Continue capturing packets

CODE:

```
import socket
S=socket.socket(socket.AF_INET, socket.SOCK_RAW,
    socket.IPPROTO_IP)
S.bind(("0.0.0.0", 0))
S.setsockopt(socket.IPPROTO_IP,socket.IP_HDRINCL,1)
print("Packet Sniffer Started...")
while True:
    data, addr = s.recvfrom(65535)
    print('Packet received from:', addr[0])
```

INPUT: Network packets

OUTPUT: Packet sniffer started...

Packet received from: 192.168.1.5

Packet received from: 192.168.1.1

Packet received from: 142.120.183.14

RESULT:

The RAW SOCKET SNIFFING PRGRM was successfully implemented and network packets was captured and displayed.

Completed

13/10/26